

TeXNote P0 リバースプロキシ・VLAN・ファイアウォール構築手順

P1 上での先行構築から P0 分離まで

2026 年 8 月 24 日

概要

本書は、TeXNote の P1 認証サーバーへ Nginx リバースプロキシと UFW を先行導入し、将来 VLAN 対応ルーターとマネージドスイッチを購入した後、公開入口を専用 Raspberry Pi 「P0」へ移す手順をまとめたものである。P1 はユーザー認証と P2 中継、P2 は TeX コンパイル、P0 は HTTPS 終端と外部公開境界を担当する。作業は段階ごとに動作確認し、問題があれば直前の構成へ戻せるように進める。

目次

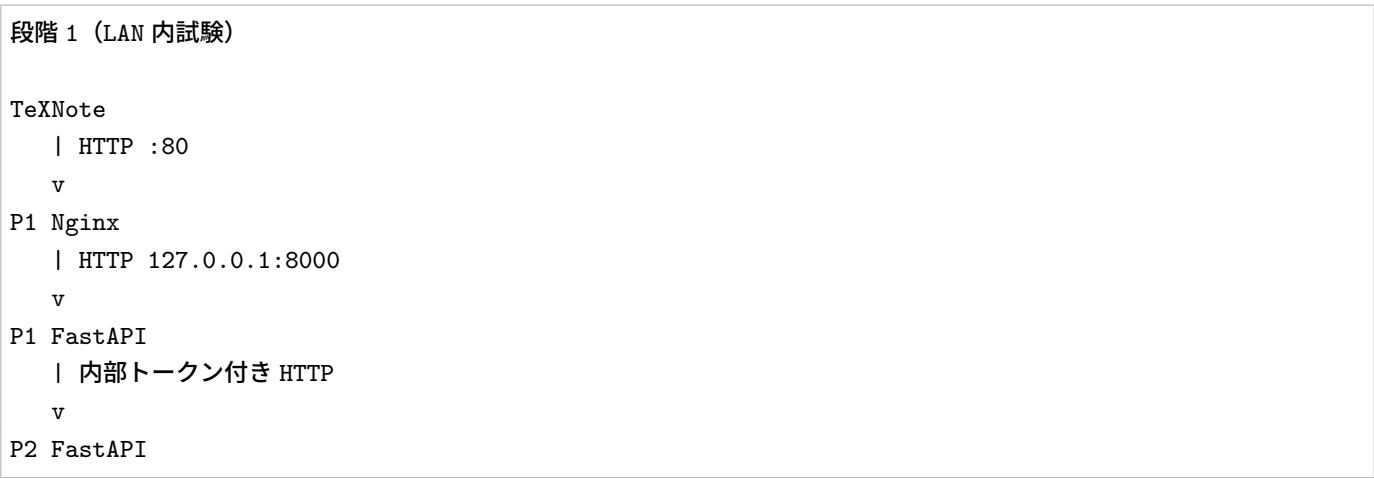
1	目的と最終構成	2
1.1	現在から最終形への移行	2
1.2	各機器の役割	2
2	作業前の原則	2
3	全体の実施順序	4
3.1	1：P1 ソースとプラン管理コマンドを更新する	4
3.2	2：管理者コマンドで plan 変更を確認する	4
3.3	3：P1 上の Nginx 経由を完成させる	5
3.4	4：P1 の UFW を適用する	5
3.5	5：P0 と VLAN へ公開境界を移す	5
3.6	6：HTTPS を設定して外部公開する	5
4	段階 1：P1 上に Nginx を構築する	6
4.1	現状確認とバックアップ	6
4.2	Nginx の導入	6
4.3	P1 用 Nginx 設定	6
4.4	サイトの有効化	7
4.5	Nginx 経由と Uvicorn 待受の確認	7
4.6	必要な場合だけ Uvicorn を localhost 限定に変更	8
4.7	TeXNote の接続先変更	8
4.8	ログ確認	9
5	段階 1：P1 の UFW を設定する	10
5.1	UFW の導入	10
5.2	有効化前に SSH と Nginx を許可	10
5.3	UFW 適用後の確認	10
5.4	再起動試験	11

6	段階 1 の切戻し	11
7	VLAN 機器の安価な購入案	12
7.1	推奨する最小構成	12
7.2	購入前に既存ルーターを確認	12
7.3	参考資料	12
8	段階 2：VLAN ネットワークを構築する	13
8.1	VLAN とアドレス設計	13
8.2	スイッチのポート設計	13
8.3	VLAN 間 ACL	13
9	段階 2：P0 を準備する	14
9.1	OS と管理ユーザー	14
9.2	Nginx と UFW の導入	14
10	段階 2：Nginx を P1 から P0 へ移す	14
10.1	設定ファイルの転送	14
10.2	P1 の待受アドレスを VLAN 側へ変更	15
10.3	P2 の接続制限	15
10.4	P0 経由の LAN 試験	15
11	段階 2：HTTPS と外部公開	16
11.1	公開前提	16
11.2	証明書取得	16
12	P0 移行後の切戻し	16
13	完了チェックリスト	17
13.1	段階 1：P1 上のリバースプロキシ	17
13.2	段階 2：P0 と VLAN	17
14	まとめ	17

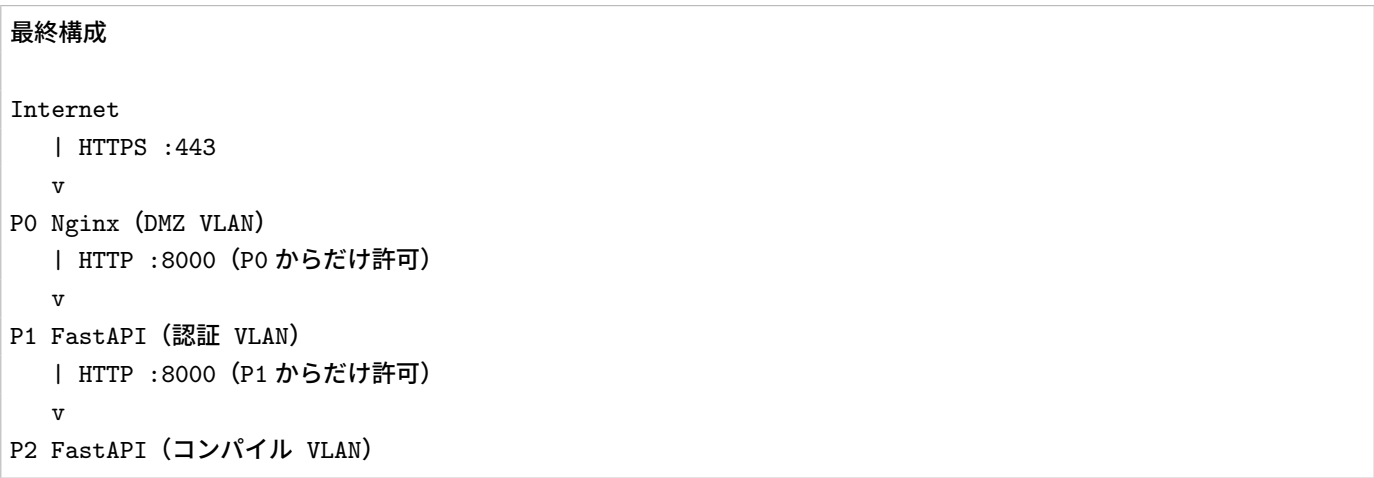
1 目的と最終構成

1.1 現在から最終形への移行

最初は P1 上で Nginx を動かし、リバースプロキシを経由するアプリ構成を先に完成させる。



VLAN 対応機器と P0 を準備した後、Nginx と HTTPS 証明書を P0 へ移す。



外部へ公開するのは P0 の TCP 443 だけである。P1、P2、SSH、TCP 8000 をインターネットへ直接公開しない。

1.2 各機器の役割

機器	役割	保持するもの・実行するもの
P0	公開境界	Nginx、HTTPS 証明書、32 MiB 制限、アクセスログ
P1	認証・API 中継	FastAPI、JWT、SQLite ユーザー DB、利用制限、P2 内部トークン
P2	TeX コンパイル	FastAPI、TeX Live、添付検査、隔離した版組処理

2 作業前の原則

1. 現行サービスの状態と設定ファイルを記録する。
2. 新しい経路が動くことを確認してから古い経路を閉じる。

3. SSH 許可規則を追加してから UFW を有効にする。
4. P1 上の Nginx が完成するまで TCP 8000 の待受方法を変更しない。
5. P0 移行が完了するまで P1 上の Nginx 設定を削除しない。
6. VLAN 間は既定拒否とし、必要な送信元、宛先、ポートだけを許可する。
7. IPv4 だけでなく IPv6 側の公開状態も確認する。

以下の現行値を前提とする。異なる場合は読み替える。

項目	現行値
P1 LAN アドレス	192.168.3.21
P2 LAN アドレス	192.168.3.4
P1 systemd サービス	tex-auth.service
P1 プログラム	/home/mat/tex-auth
P1 仮想環境	/home/mat/tex-auth/.auth-env
P1 FastAPI ポート	TCP 8000
Nginx LAN 試験ポート	TCP 80
公開時 HTTPS ポート	TCP 443
HTTP 要求上限	P0、P1、P2 すべて 32 MiB

3 全体の実施順序

本書と P1 構築手順書に分散している作業を、実機へ適用する順番にまとめる。各段階では、確認項目がすべて成功してから次へ進む。失敗した場合は次の段階へ進まず、その段階で退避した設定へ戻す。

3.1 1: P1 ソースとプラン管理コマンドを更新する

最初に、P1 の現行プログラムと SQLite DB を退避する。P1 上で実行する。

```
cd /home/mat/tex-auth
backup_dir=/home/mat/tex-auth-backup-before-plan
mkdir -p "$backup_dir"
cp -p main.py auth.py database.py tex_service.db "$backup_dir"/
test ! -f plans.py || cp -p plans.py "$backup_dir"/
test ! -d tools || cp -a tools "$backup_dir/"
```

管理用 Mac の TeXNote リポジトリ直下から、秘密値と DB を含めずに現行ソースを転送する。

```
scp Server/P1/progs/main.py \
    Server/P1/progs/auth.py \
    Server/P1/progs/database.py \
    Server/P1/progs/plans.py \
    mat@192.168.3.21:/home/mat/tex-auth/

scp -r Server/P1/progs/tools \
    mat@192.168.3.21:/home/mat/tex-auth/
```

P1 上で構文を検査してからサービスを再起動する。既存 DB に対して tools.init.db を実行し直してはならない。

```
cd /home/mat/tex-auth
source .auth-env/bin/activate
python -m py_compile *.py tools/*.py
sudo systemctl restart tex-auth.service
systemctl status tex-auth.service --no-pager -l
journalctl -u tex-auth.service -n 100 --no-pager
```

3.2 2: 管理者コマンドで plan 変更を確認する

既存ユーザーを例として Standard へ変更し、DB の結果を確認する。メールアドレスは実際の登録ユーザーへ置き換える。

```
cd /home/mat/tex-auth
source .auth-env/bin/activate
python -m tools.change_plan user@example.com standard
python -m tools.check02 user@example.com
```

change_plan が変更前と変更後の plan を表示し、check02 にも standard が表示されることを確認する。変更後の plan は要求ごとに DB から読み込むため、plan 変更だけを理由に P1 を再起動する必要はない。現段階では/me を利用したプラン変更 UI は作らず、管理者コマンドだけを正式な変更経路とする。

3.3 3：P1 上の Nginx 経由を完成させる

本書の「段階 1：P1 上に Nginx を構築する」に従う。Uvicorn の待受を変更する前に、Nginx 経由の `/login`、`/me`、`/compile` を確認する。同時に、Nginx 経由の `/compile-test`、`/docs`、`/openapi.json` が 404 になることを確認する。

Uvicorn が `127.0.0.1:8000` だけで待ち受けていることを確認する。実機が既にこの状態なら設定編集、`daemon-reload`、待受変更を目的とした P1 再起動は行わない。`0.0.0.0:8000` または `:::8000` で待ち受けている場合だけ、Nginx 経由の正常動作を確認した後に `127.0.0.1:8000` 限定へ変更する。`/compile-test` は削除せず、P1 へ SSH 接続した管理者が `127.0.0.1:8000/compile-test` を使う保守 API として残す。

3.4 4：P1 の UFW を適用する

SSH と Nginx への許可規則を先に追加してから UFW を有効にする。別ターミナルから新しい SSH 接続が成功するまで、作業中の SSH 接続を閉じない。LAN 上の端末から P1 の TCP 8000 へ直接接続できず、P1 自身から `127.0.0.1:8000` へ接続できることを確認する。その後 P1 を再起動し、Nginx、P1、UFW が自動復旧することを確認する。

3.5 5：P0 と VLAN へ公開境界を移す

段階 1 が安定してから、本書の「段階 2：P0 を準備する」以降へ進む。P0 の Nginx は `/login`、`/me`、`/compile` だけを P1 へ転送する。P1 の Uvicorn は認証 VLAN 側アドレスだけで待ち受け、P1 の UFW は P0 からの TCP 8000 だけを許可する。P2 の UFW は P1 からの TCP 8000 だけを許可する。

P0 経由の通常版組まで成功し、P0 から P2 へ直接接続できないことを確認するまで、P1 上の Nginx 設定を削除しない。問題が発生した場合は、本書の切戻し手順で段階 1 へ戻す。

3.6 6：HTTPS を設定して外部公開する

最後に P0 へ HTTPS 証明書を設定する。ルーターでインターネットから転送するのは P0 の TCP 443 だけとし、TCP 22、80、8000、P1、P2 を直接公開しない。外部から `/compile-test`、`/docs`、`/openapi.json` が 404 となり、アプリの `/login`、`/me`、`/compile` だけが HTTPS で利用できることを最終確認する。

4 段階 1：P1 上に Nginx を構築する

4.1 現状確認とバックアップ

P1 上で実行する。

```
systemctl status tex-auth.service --no-pager -l
systemctl is-enabled tex-auth.service
systemctl is-active tex-auth.service
sudo ss -ltnp | grep -E ':8000|:80|:443'
```

現在のサービスファイルを退避する。

```
sudo cp -p \
/etc/systemd/system/tex-auth.service \
/etc/systemd/system/tex-auth.service.before-nginx
```

4.2 Nginx の導入

```
sudo apt update
sudo apt install nginx
nginx -v
sudo systemctl status nginx --no-pager
```

初期ページを確認する。

```
curl -I http://192.168.3.21/
```

4.3 P1 用 Nginx 設定

```
sudoedit /etc/nginx/sites-available/texnote-p1
```

次の内容を設定する。proxy_pass の末尾にはスラッシュを付けず、/login、/me、/compile の完全一致だけを P1 へ渡す。location /でその他を 404 にするため、保守用の/compile-test、/docs、/openapi.json は Nginx 経由で公開されない。

```
server {
    listen 80;
    listen [::]:80;

    server_name _;
    server_tokens off;

    client_max_body_size 32m;
    client_body_timeout 30s;

    access_log /var/log/nginx/texnote-p1-access.log;
    error_log /var/log/nginx/texnote-p1-error.log;

    location = /login {
```

```

        proxy_pass http://127.0.0.1:8000;
        include /etc/nginx/proxy_params;
        proxy_connect_timeout 5s;
        proxy_send_timeout 30s;
        proxy_read_timeout 30s;
    }

    location = /me {
        proxy_pass http://127.0.0.1:8000;
        include /etc/nginx/proxy_params;
        proxy_connect_timeout 5s;
        proxy_send_timeout 30s;
        proxy_read_timeout 30s;
    }

    location = /compile {
        proxy_pass http://127.0.0.1:8000;
        include /etc/nginx/proxy_params;
        proxy_connect_timeout 5s;
        proxy_send_timeout 120s;
        proxy_read_timeout 120s;
    }

    location / {
        return 404;
    }
}

```

P0、P1、P2 の要求上限を 32 MiB へ統一する。Nginx の既定値は 1 MiB であるため、`client_max_body_size 32m;` を省略しない。

4.4 サイトの有効化

```

sudo ln -s \
    /etc/nginx/sites-available/texnote-p1 \
    /etc/nginx/sites-enabled/texnote-p1
sudo unlink /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl enable nginx

```

`nginx -t` が `syntax is ok`、`test is successful` を返してから reload する。

4.5 Nginx 経由と Uvicorn 待受の確認

最初に、現在の `systemd` 設定と実際の待受アドレスを確認する。

```

systemctl cat tex-auth.service
sudo ss -ltnp | grep ':8000'

```

実機が既に `127.0.0.1:8000` だけで待ち受けている場合は、それが正しい現行状態である。待受設定を変更せず、その状態のまま Nginx 経由を確認する。


```
curl --fail-with-body http://127.0.0.1:8000/openapi.json >/dev/null
test "$(curl -sS -o /dev/null -w '%{http_code}' \
  http://192.168.3.21/openapi.json)" = 404
test "$(curl -sS -o /dev/null -w '%{http_code}' -X POST \
  http://192.168.3.21/compile-test)" = 404
```

最初の要求は P1 自身の保守 API を直接確認する。後の 2 要求は Nginx が公開対象外のパスを P1 へ転送せず、認証処理より前に 404 を返すことを確認する。

登録済みユーザーでログインする。

```
curl -sS -X POST http://192.168.3.21/login \
-H 'Content-Type: application/json' \
-d '{
  "email": "<登録済みメールアドレス>",
  "password": "<パスワード>"
}'
```

4.6 必要な場合だけ Uvicorn を localhost 限定に変更

前項の確認で 127.0.0.1:8000 だけが表示された場合、この項の編集操作はすべて省略する。0.0.0.0:8000 または [::]:8000 が表示された場合に限り、Nginx 経由のログインが成功してから変更する。

```
sudoedit /etc/systemd/system/tex-auth.service
```

ExecStart を 1 行で次のようにする。

```
ExecStart=/home/mat/tex-auth/.auth-env/bin/uvicorn main:app --host 127.0.0.1 --port 8000
```

設定を編集した場合だけ反映する。

```
sudo systemctl daemon-reload
sudo systemctl restart tex-auth.service
systemctl status tex-auth.service --no-pager -l
journalctl -u tex-auth.service -n 100 --no-pager
```

編集を省略した場合も含め、最後に待受状態を確認する。

```
sudo ss -ltnp | grep -E ':8000|:80|:443'
```

期待する待受状態は次のとおりである。

0.0.0.0:80	nginx
[::]:80	nginx
127.0.0.1:8000	uvicorn

0.0.0.0:8000 が残っているはいけない。

4.7 TeXNote の接続先変更

macOS 版、iPad 版、iPhone 版の P1 URL を次のように変更する。

```
変更前: http://192.168.3.21:8000  
変更後: http://192.168.3.21
```

80 番は HTTP の標準ポートなので省略できる。各アプリでログイン、/me、通常版組、画像付き版組、TeX エラー表示を確認する。

4.8 ログ確認

```
sudo tail -n 100 /var/log/nginx/texnote-p1-access.log  
sudo tail -n 100 /var/log/nginx/texnote-p1-error.log  
journalctl -u nginx -n 100 --no-pager
```

リアルタイム監視は次のとおり。

```
sudo tail -f \  
  /var/log/nginx/texnote-p1-access.log \  
  /var/log/nginx/texnote-p1-error.log
```

終了は Ctrl+C で行う。

5 段階 1：P1 の UFW を設定する

5.1 UFW の導入

```
sudo apt update
sudo apt install ufw
ufw --version
sudo systemctl status ufw --no-pager
```

5.2 有効化前に SSH と Nginx を許可

LAN が 192.168.3.0/24 の場合の例を示す。現在の SSH 接続を閉じずに作業する。

```
sudo ufw allow from 192.168.3.0/24 to any port 22 proto tcp
sudo ufw allow from 192.168.3.0/24 to any port 80 proto tcp
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw deny 8000/tcp
sudo ufw show added
```

内容を確認してから有効化する。

```
sudo ufw enable
sudo ufw status verbose
sudo ufw status numbered
```

Mac の別ターミナルから新しい SSH 接続を試す。

```
ssh mat@192.168.3.21
```

新しい SSH 接続に成功するまで、元の SSH 接続を終了しない。

5.3 UFW 適用後の確認

Nginx 経由の/login が P1 へ到達する必要がある。空 JSON に対する 422 は、Nginx ではなく P1 の入力検証が応答したことを示す。

```
test "$(curl -sS -o /dev/null -w '%{http_code}' -X POST \
-H 'Content-Type: application/json' -d '{}') \
http://192.168.3.21/login)" = 422
test "$(curl -sS -o /dev/null -w '%{http_code}' \
http://192.168.3.21/openapi.json)" = 404
```

LAN から Uvicorn へ直接接続する次のコマンドは失敗するのが正常である。

```
curl --connect-timeout 5 http://192.168.3.21:8000/docs
```

P1 自身から localhost の Uvicorn へは接続できる。

```
curl --fail-with-body \
http://127.0.0.1:8000/openapi.json >/dev/null
```

5.4 再起動試験

```
sudo reboot
```

再接続後に確認する。

```
systemctl is-active nginx
systemctl is-enabled nginx
systemctl is-active tex-auth.service
systemctl is-enabled tex-auth.service
sudo ufw status verbose
sudo ss -ltnp | grep -E ':8000|:80|:443'
curl --fail-with-body http://127.0.0.1:8000/openapi.json >/dev/null
```

6 段階 1 の切戻し

Nginx 経由で問題が発生した場合、バックアップした systemd 設定を戻す。

```
sudo cp -p \
  /etc/systemd/system/tex-auth.service.before-nginx \
  /etc/systemd/system/tex-auth.service
sudo systemctl daemon-reload
sudo systemctl restart tex-auth.service
sudo systemctl disable --now nginx
```

UFW の 8000 拒否規則を番号で確認して削除する。

```
sudo ufw status numbered
sudo ufw delete <8000/tcp 拒否規則の番号>
```

TeXNote の URL を従来値へ戻す。

```
http://192.168.3.21:8000
```

7 VLAN 機器の安価な購入案

7.1 推奨する最小構成

2026 年 8 月時点で日本国内から入手しやすい構成例を示す。価格と販売状況は購入時に再確認する。

機器	候補	価格目安
VLAN ルーター	TP-Link ER605 V2	約 9,500～11,500 円
5 ポート VLAN スイッチ	TP-Link TL-SG105E	約 3,000～3,500 円
8 ポート VLAN スイッチ	TP-Link TL-SG108E	約 5,000～6,500 円

ER605 は 802.1Q VLAN、複数ネットワーク、送信元・宛先 IP による ACL、ファイアウォールに対応する。TL-SG105E/TL-SG108E は 802.1Q タグ VLAN に対応する。

P0、P1、P2、管理用 Mac、ルーターとの trunk で 5 ポートをすべて使用するため、長期運用では 8 ポート版が扱いやすい。最小費用を優先する場合は 5 ポート版でも構築できる。

7.2 購入前に既存ルーターを確認

既存ルーターが次のすべてに対応する場合、ER605 を追加せず VLAN スイッチだけで構築できる。

- 802.1Q VLAN
- VLAN ごとの IP インターフェースと DHCP
- VLAN 間 ACL
- 送信元、宛先、プロトコル、ポートによる許可・拒否
- ポート転送

家庭用ルーターの「VLAN」が IPTV 専用機能だけの場合は使用できない。

7.3 参考資料

- ER605 : <https://www.tp-link.com/jp/business-networking/vpn-router/er605/v2/>
- TL-SG105E : <https://www.tp-link.com/jp/business-networking/easy-smart-switch/tl-sg105e/>
- Omada 複数ネットワークと ACL : <https://www.tp-link.com/jp/support/faq/3061/>
- Omada router-on-a-stick : <https://www.tp-link.com/jp/support/faq/3307/>
- Nginx proxy module : https://nginx.org/en/docs/http/nginx_http_proxy_module.html
- Nginx client_max_body_size : https://nginx.org/en/docs/http/nginx_http_core_module.html#client_max_body_size
- Raspberry Pi UFW : <https://www.raspberrypi.com/documentation/security/ufw.html>

8 段階 2：VLAN ネットワークを構築する

8.1 VLAN とアドレス設計

VLAN	用途	ネットワーク	固定アドレス例
20	DMZ	192.168.20.0/24	P0 192.168.20.10
30	認証	192.168.30.0/24	P1 192.168.30.10
40	コンパイル	192.168.40.0/24	P2 192.168.40.10
99	管理	192.168.99.0/24	管理用 Mac

VLAN ルーターの各ゲートウェイを 192.168.X.1 とする。

8.2 スイッチのポート設計

ポート	接続先	VLAN 設定	PVID
1	VLAN ルーター	VLAN 20/30/40/99 Tagged	99
2	P0	VLAN 20 Untagged	20
3	P1	VLAN 30 Untagged	30
4	P2	VLAN 40 Untagged	40
5	管理用 Mac	VLAN 99 Untagged	99

P0、P1、P2 は通常のタグなし Ethernet を使用し、VLAN タグはスイッチ側で付け外しする。

8.3 VLAN 間 ACL

許可規則を拒否規則より先に置く。

送信元	宛先	ポート	動作
管理 VLAN 99	P0/P1/P2	TCP 22	許可
P0	P1	TCP 8000	許可
P1	P2	TCP 8000	許可
DMZ VLAN 20	認証 VLAN 30	上記以外	拒否
DMZ VLAN 20	コンパイル VLAN 40	すべて	拒否
認証 VLAN 30	コンパイル VLAN 40	上記以外	拒否
サーバー VLAN	管理 VLAN 99	すべて	拒否

OS 更新に必要な DNS、NTP、HTTPS の外向き通信は動作確認しながら許可する。二重ルーター構成では、サーバー VLAN から既存家庭内 LAN への通信も拒否する。IPv6 を使用する場合は IPv6 にも同等の規則を設定し、未設定の間はサーバー VLAN へのグローバル IPv6 配布を止める。

9 段階 2：P0 を準備する

9.1 OS と管理ユーザー

P0 へ 64-bit Raspberry Pi OS Lite または Debian を導入し、固定アドレス 192.168.20.10 を設定する。初期設定後、更新する。

```
sudo apt update
sudo apt full-upgrade
sudo reboot
```

管理用 SSH は VLAN 99 からだけ許可し、インターネットへ 22 番を転送しない。

9.2 Nginx と UFW の導入

```
sudo apt install nginx ufw
sudo ufw allow from 192.168.99.0/24 to any port 22 proto tcp
sudo ufw allow 443/tcp
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose
```

公開前の LAN 試験で 80 番が必要な期間だけ、管理 VLAN または LAN から限定して許可する。

```
sudo ufw allow from 192.168.99.0/24 to any port 80 proto tcp
```

10 段階 2：Nginx を P1 から P0 へ移す

10.1 設定ファイルの転送

リポジトリで管理する P0 用設定を Mac から P0 へ転送する。この設定は P1 の VLAN アドレスを転送先とし、公開 API を 3 パスへ限定している。秘密鍵はこの時点では含まれない。

```
scp Server/P0/progs/texnote-p0.nginx \
mat@192.168.20.10:/home/mat/
```

P0 上で配置する。

```
sudo install -o root -g root -m 644 \
/home/mat/texnote-p0.nginx \
/etc/nginx/sites-available/texnote-p0
```

配置後、location = /login、location = /me、location = /compile だけに proxy_pass があり、location / が 404 を返すことを確認する。

設定名を有効化する。

```
sudo ln -s \
/etc/nginx/sites-available/texnote-p0 \
/etc/nginx/sites-enabled/texnote-p0
sudo unlink /etc/nginx/sites-enabled/default
```

```
sudo nginx -t
sudo systemctl reload nginx
```

10.2 P1 の待受アドレスを VLAN 側へ変更

P1 の `tex-auth.service` を編集する。

```
ExecStart=/home/mat/tex-auth/.auth-env/bin/uvicorn main:app --host 192.168.30.10 --port 8000
```

P1 の UFW に以前追加した 8000 番の全面拒否を削除し、P0 だけを許可する。番号は実機で確認する。

```
sudo ufw status numbered
sudo ufw delete <8000/tcp 全面拒否規則の番号>
sudo ufw allow from 192.168.20.10 to any port 8000 proto tcp
sudo ufw status numbered
```

systemd 設定を反映する。

```
sudo systemctl daemon-reload
sudo systemctl restart tex-auth.service
systemctl status tex-auth.service --no-pager -l
sudo ss -ltnp | grep ':8000'
```

10.3 P2 の接続制限

P2 は P1 からの TCP 8000 だけを許可する。

```
sudo apt install ufw
sudo ufw allow from 192.168.99.0/24 to any port 22 proto tcp
sudo ufw allow from 192.168.30.10 to any port 8000 proto tcp
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose
```

P1 の環境ファイルで P2 の新アドレスを設定する。

```
TEXNOTE_P2_TYPESET_URL=http://192.168.40.10:8000/v1/typeset
```

P1 を再起動する。

```
sudo systemctl restart tex-auth.service
```

10.4 P0 経由の LAN 試験

管理用 Mac から、P0 が公開対象外のパスを拒否することを先に確認する。

```
test "$(curl -sS -o /dev/null -w '%{http_code}' \
  http://192.168.20.10/openapi.json)" = 404
test "$(curl -sS -o /dev/null -w '%{http_code}' -X POST \
```



```
http://192.168.20.10/compile-test)" = 404
```

続いて P0 経由でログイン、/me、実際の画像付き/compileを確認する。/compile-test による P1-P2 間の保守確認が必要な場合は P1 へ SSH 接続し、P1 自身の 127.0.0.1:8000/compile-test を使用する。P0 から P2 へ直接接続できず、P0 から P1 の TCP 8000 だけへ接続できることも確認する。

11 段階 2：HTTPS と外部公開

11.1 公開前提

HTTPS 証明書を取得するには、通常は所有するドメイン名、DNS 設定、外部から P0 へ到達できる経路が必要である。契約回線が CGNAT の場合は通常のポート転送が使えないことがあるため、公開前に確認する。

ルーターの転送は TCP 443 だけとする。

```
Internet TCP 443 -> P0 192.168.20.10 TCP 443
```

TCP 22、80、8000 を恒常的にインターネットへ転送しない。

11.2 証明書取得

ドメインと DNS を準備した後、P0 へ Certbot の Nginx 連携を導入する例を示す。

```
sudo apt update
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d texnote.example.com
```

texnote.example.com は実際のドメインへ置き換える。取得後に自動更新を確認する。

```
systemctl status certbot.timer --no-pager
sudo certbot renew --dry-run
sudo nginx -t
```

TeXNote の P1 URL を HTTPS へ変更する。

```
https://texnote.example.com
```

12 P0 移行後の切戻し

P0 または VLAN で問題が発生した場合、外部公開を停止して LAN 内の P1 Nginx へ戻す。

1. ルーターの TCP 443 転送を停止する。
2. P0 Nginx を停止する。
3. P1 Unicorn を 127.0.0.1:8000 へ戻す。
4. P1 上の Nginx を再度有効化する。
5. TeXNote の接続先を P1 の LAN URL へ戻す。

```
# P0
sudo systemctl stop nginx

# P1
sudoedit /etc/systemd/system/tex-auth.service
```

```
# ExecStart の host を 127.0.0.1 へ戻す
sudo systemctl daemon-reload
sudo systemctl restart tex-auth.service
sudo systemctl enable --now nginx
sudo nginx -t
```

13 完了チェックリスト

13.1 段階 1：P1 上のリバースプロキシ

- ☐ Nginx が enabled かつ active である。
- ☐ Nginx の要求上限が 32 MiB である。
- ☐ P1 Uvicorn が 127.0.0.1:8000 だけで待ち受けている。
- ☐ LAN から TCP 8000 へ直接接続できない。
- ☐ Nginx 経由で login、me、compile が成功する。
- ☐ UFW 有効化後も新しい SSH 接続が成功する。
- ☐ P1 再起動後も Nginx と P1 が自動復旧する。

13.2 段階 2：P0 と VLAN

- ☐ P0、P1、P2、管理用端末が別 VLAN へ分離された。
- ☐ P0 から P1 の TCP 8000 だけが許可された。
- ☐ P1 から P2 の TCP 8000 だけが許可された。
- ☐ P0 から P2 へ直接接続できない。
- ☐ インターネットへ公開されるのは P0 の TCP 443 だけである。
- ☐ SSH は管理 VLAN からだけ許可される。
- ☐ IPv6 にも同等の制限がある、またはサーバー VLAN で無効化されている。
- ☐ HTTPS 証明書の自動更新試験が成功した。
- ☐ P0、P1、P2 再起動後に全経路が自動復旧する。
- ☐ 段階 1 へ戻す切戻し手順を確認した。

14 まとめ

最初に P1 上で Nginx を構築すると、TeXNote、P1 FastAPI、P2 FastAPI の処理を変えずにリバースプロキシ経由の動作を確認できる。次に VLAN 機器と P0 を追加し、Nginx の `proxy_pass` を `localhost` から P1 の内部アドレスへ変更することで、公開境界だけを P0 へ移せる。安全性を決めるのは機器を分けることだけではなく、P0 から P1、P1 から P2 という必要最小限の通信だけを ACL と各ホストの UFW で許可することである。